

#05 Number Theory I

김성수

4thrunyul@gmail.com

0. 목차



1. 문제 풀이 전략

- 디버깅

2. 정수론

- 최대공약수(GCD)와 최소공배수(LCM)
- 유클리드 호제법
- 소수 판별
- 에라토스테네스의 체
- 나머지 연산의 분배 법칙

1. 문제 풀이 전략: 순서



문제 파악 - 수도 코드 작성 - 구현 - 디버깅 - 제출 - 피드백

* 굵은 표시가 되어있는 부분은 이번 시간에 주로 다룰 것을 표시한 것이지, 중요도를 나타낸 게 아님

1. 문제풀이 전략 : 디버깅



디버깅(Debugging)

- 정의 : 소프트웨어의 결함(Bug)을 식별하고 그 원인을 분석하여 수정하는 과정
- IDE 내부의 디버깅 기능을 사용하거나 직접 입출력을 지정하여 확인하는 출력 디버깅 방법이 있음.
- 출력 디버깅
 - 로직의 흐름을 순차적으로 추적 : 함수 정의 → 초기화 → 연산 → 출력 단계별로 중간값이 예상과 맞는지 확인
 - 경계값 점검 : 루프의 시작과 끝, 조건문의 분기점에서 상태가 변하는 결정적 지점에서 출력

1. 문제풀이 전략 : 출력 디버깅

```
1  #include <iostream>
2
3  long long arr[100001];
4
5  using namespace std;
6
7  int main() {
8      long long n, k, result;
9      long long total_Distance = 0;
10
11     cin >> n >> k;
12
13     for(int i = 0; i < n; i++) {
14         cin >> arr[i];
15
16         total_Distance += arr[i];
17     }
```

```
19     if(k >= total_Distance) {
20         k -= total_Distance;
21
22         for(int i = n - 1; i >= 0; i--) {
23             if(k < arr[i]) {
24                 result = i + 1;
25
26                 break;
27             }
28             else {
29                 k -= arr[i];
30             }
31         }
32     }
```

```
33     else {
34         for(int i = 0; i < n; i++) {
35             if(k < arr[i]) {
36                 result = i + 1;
37
38                 break;
39             }
40             else {
41                 k -= arr[i];
42             }
43         }
44     }
45
46     cout << result;
47
48     return 0;
49 }
```

비대화된 코드와 로직은 디버깅을 어떻게 해야 할까?

1. 문제풀이 전략 : 출력 디버깅

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  int main() {
7      vector<int> v = {3, 7, 2, 9, 4};
8      int maxVal = v[0];
9
10     // 1. 검증하려는 로직만 실행
11     for (int i = 1; i < v.size(); i++) {
12         if (v[i] > maxVal) {
13             maxVal = v[i];
14         }
15     }
16
17     // 2. 검증하려는 로직을 제외한 모든 로직 주석화
18     // if(maxVal % 2 == 0) maxVal *= 2;
19     // else maxVal *= 5;
20
21     // 3. 예측되는 출력값과 실제 출력값이 같은지 확인
22     cout << maxVal << "\n";
23
24     return 0;
25 }
```

코드 전체 설명

- 벡터에서 최댓값을 찾은 뒤, 해당 값이 짝수면 2를 곱하고 홀수면 5를 곱하는 코드

로직 실행 순서대로 체크

- 검증하려는 로직 이후에 실행되는 모든 로직을 주석화
- 검증하려는 로직의 예측 출력값과 실제 출력값이 같은지 확인

출력 디버깅

- 벡터에서 최댓값을 찾는 로직만 수행
- 해당 값이 짝수인지 홀수인지 판단하는 로직과 2, 5를 곱하는 코드를 주석화하였음
- 만약 검증하려는 로직이 문제가 없다면, 바로 다음 로직의 주석을 풀고 다시 검증 반복

1. 문제풀이 전략 : 디버깅, 대표적인 오류



1. 변수값 초기화 안함

- 변수를 선언만 하고 초기값을 할당하지 않아 쓰레기 값(Garbage Value)이 남은 상태
- 똑같은 입력을 넣고 실행하였음에도 불구하고 출력값이 바뀜(실행마다 결과가 달라짐)
- 결과값으로 예상치 못한 큰 수 출력
- 초기화되지 않은 값이 배열의 인덱스나 포인터로 사용되면 RE(Runtime Error) 발생

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n, m;
7
8      cin >> n;
9
10     cout << n + m;
11
12     return 0;
13 }
```

1. 문제풀이 전략 : 디버깅, 대표적인 오류



2. 배열 범위 초과(Out of Bounds)

- 선언된 배열의 크기를 인덱스에 접근하여 데이터를 읽거나 쓰는 상태
- Segmentation Fault : 할당되지 않은 메모리 영역을 참조할 때 프로그램이 즉시 강제 종료됨.
- 프로그램이 종료되지 않더라도, 인접한 다른 변수의 값을 오염시킴

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      char arr[5] = {'a', 'b', 'c', 'd', 'e'};
7
8      // arr[5]에서 Out of Bounds 발생
9      for(int i = 0; i <= 5; i++) {
10         cout << arr[i];
11     }
12
13     return 0;
14 }
```

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  int main() {
7      vector<int> v;
8
9      cin >> v[0];
10
11     cout << v[0];
12
13     // vector v가 선언만 되어있고
14     // 칸이 생성되어 있지 않음
15     // 즉, 접근하면 프로그램이 정지됨
16
17     return 0;
18 }
```

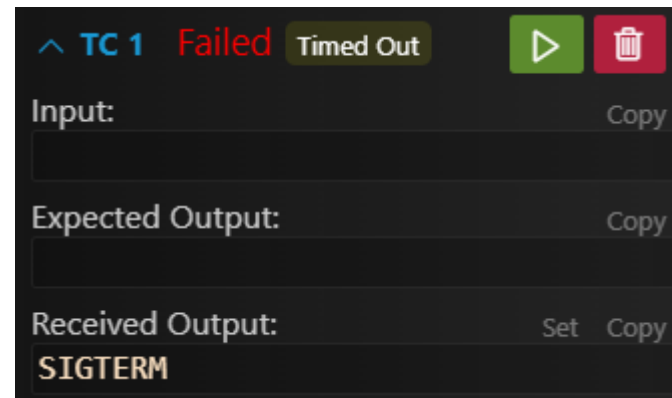

1. 문제풀이 전략 : 디버깅, 대표적인 오류



3. 무한 루프

- 반복문의 종료 조건이 충족되지 못해 프로그램이 영원히 실행되는 상태
- 증감 연산 누락, 조건문 내 오류 등 원인이 다양함

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n = 1;
7
8      while(n > 0) {
9          cout << n;
10
11         n++;
12     }
13
14     return 0;
15 }
```



1. 문제풀이 전략 : 디버깅, 대표적인 오류



4. 정수 오버플로우

- 정수형 변수가 표현할 수 있는 최대 범위를 초과하여 값이 가장 작은 값으로 순환하는 상태
- 양수끼리 더하거나 곱했는데 음수로 출력 됨
- 조건문 실패 : $a + b > k$ 같은 비교 연산을 진행하려 하였지만 $a + b$ 가 오버플로우가 되면 조건문이 잘못 작동함
- long long 자료형을 써야하지만 int 자료형을 쓴 경우

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n, m, k; // int 약 21억까지 표현)
7
8      n = 1500000000; // 15억
9      m = 1000000000; // 10억
10     k = 2000000000; // 20억
11
12     if(k < n + m) cout << (n + m) % k;
13     else cout << "OverFlow";
14
15     return 0;
16 }
```

Input:

Copy

Expected Output:

Copy

OverFlow

2. 정수론: 나머지 연산



나머지 연산(mod 연산)

- 정수 a , n 이 있을 때, $a \bmod n$ 은 a 를 n 으로 나뉘었을 때의 나머지
- $7 \bmod 3 = 1$
- $10 \bmod 5 = 0$
- $3 \bmod 10 = 3$
- $a \bmod n$ 의 결과가 r 이라면 0 이상 $n - 1$ 이하의 정수가 되기에 값의 범위를 제한하는 특성이 있음.

2. 정수론: 나머지 연산



나머지 연산의 분배 법칙

- 나머지 연산은 덧셈, 뺄셈, 곱셈의 분배 법칙이 성립함.
- 덧셈
 - $(A + B) \% C = (A \% C + B \% C) \% C$
 - $(20 + 6) \% 3 = 26 \% 3 = 2$
 - $(20 \% 3 + 6 \% 3) \% 3 = (2 + 0) \% 3 = 2$
- 뺄셈
 - $(A - B) \% C = (A \% C - B \% C) \% C$
 - $(20 - 6) \% 3 = 14 \% 3 = 2$
 - $(20 \% 3 - 6 \% 3) \% 3 = (2 - 0) \% 3 = 2$
- 곱셈
 - $(A * B) \% C = (A \% C * B \% C) \% C$
 - $(20 * 6) \% 3 = 120 \% 3 = 0$
 - $(20 \% 3 * 6 \% 3) \% 3 = (2 * 0) \% 3 = 0$

2. 정수론: 나머지 연산



```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      long long sum = 1;
7
8      for(int i = 1; i <= 50; i++) {
9          sum *= i;
10     }
11
12     cout << sum % 1234567891;
13
14     return 0;
15 }
```

Input: Copy

Expected Output: Copy

-1076673018

50!를 1234567891로 나눈 나머지를 구하려고 한다.

해당 코드가 오류가 발생하는 이유는?

- 21!에서 이미 long long 범위를 벗어나서 overflow 문제가 발생함

오류를 바로잡고 원하는 결과를 얻기 위해 수정해야 할 로직은?

2. 정수론: 나머지 연산

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      long long sum = 1;
7
8      for(int i = 1; i <= 50; i++) {
9          sum = (sum * i) % 1234567891;
10     }
11
12     cout << sum;
13
14     return 0;
15 }
```

Input: Copy

Expected Output: Copy

1068536508

50!를 1234567891로 나눈 나머지를 구하려고 한다.

수정된 로직

- 연산이 완료된 시점에서 나머지 연산을 수행하는 것이 아니라 연산이 수행 되는 도중에 계속 나머지 연산을 수행함.
- 나머지 연산의 분배 법칙에 근거해 원하는 결과값을 얻을 수 있음.

2. 정수론: 실습



실습

- [33046 Alea lacta Est](#)
- [15829 Hashing](#)

2. 정수론: 최대공약수와 최소공배수

약수(Divisor)

- 두 정수 a 와 b 에 대해 $a = bk$ 를 만족하는 정수 k 가 존재할 때, ' b 는 a 의 약수다'라고 정의함.

최대공약수(GCD, Greatest Common Divisor)

- 둘 이상의 정수들의 공통된 약수 중 가장 큰 양수.

최소공배수(LCM, Least Common Multiple)

- 둘 이상의 정수들의 공통된 배수 중에서 가장 작은 양수

2. 정수론: 최대공약수와 최소공배수

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7
8      cin >> n;
9
10     for(int i = 1; i <= n; i++) {
11         if(n % i == 0) cout << i << " ";
12     }
13
14     return 0;
15 }
```

Input: Copy

10

Expected Output: Copy

1 2 5 10

약수 판별

- 어떤 정수 N이 있을 때 정수 i로 나누어 떨어진다면 약수
- 해당 코드에서는 1 ~ N까지 순회를 통해 나머지가 0이라면 출력

2. 정수론: 최대공약수와 최소공배수

소인수분해

- 1보다 큰 자연수를 소수(Prime Number)들의 곱으로만 나타낸 것
- $2 \sim \sqrt{n}$ 까지 나머지 연산을 실행하여 나머지가 0이라면 출력
- 약수는 한 쌍으로 존재함($16 = 1 * 16 = 2 * 8 = 4 * 4$)
- 두 인수가 같이 $\sqrt{n}$ 보다 클 수 없음.

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7
8      cin >> n;
9
10     for(int i = 2; i * i <= n; i++) {
11         while(n % i == 0) {
12             cout << i << " ";
13
14             n /= i;
15         }
16     }
17
18     if(n > 1) cout << n;
19
20     return 0;
21 }
    
```

Input:

60

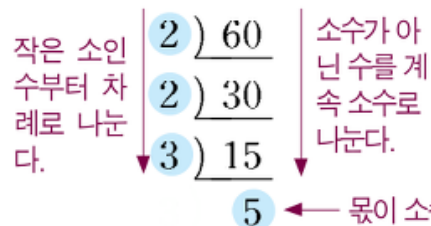
Copy

Expected Output:

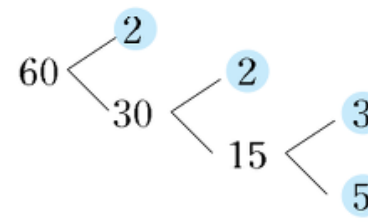
2 2 3 5

Copy

[방법 1]



[방법 2]



나눈 소수들과 마지막 몫을 곱으로 나타낸다.

소인수분해한 결과

$$60 = 2 \times 2 \times 3 \times 5 = 2^2 \times 3 \times 5$$

60의 소인수 : 2, 3, 5

2. 정수론: 최대공약수와 최소공배수

```

1  #include <iostream>
2  #include <numeric>
3
4  using namespace std;
5
6  int main() {
7      int a, b, result_gcd, result_lcm;
8
9      cin >> a >> b;
10
11     result_gcd = gcd(a, b);
12     result_lcm = lcm(a, b);
13     //result_lcm = a * b / gcd(a, b);
14     //ex) 12 * 18 / 6 = 36;
15
16     cout << result_gcd << " " << result_lcm;
17
18     return 0;
19 }

```

Input:

12 18

Copy

Expected Output:

6 36

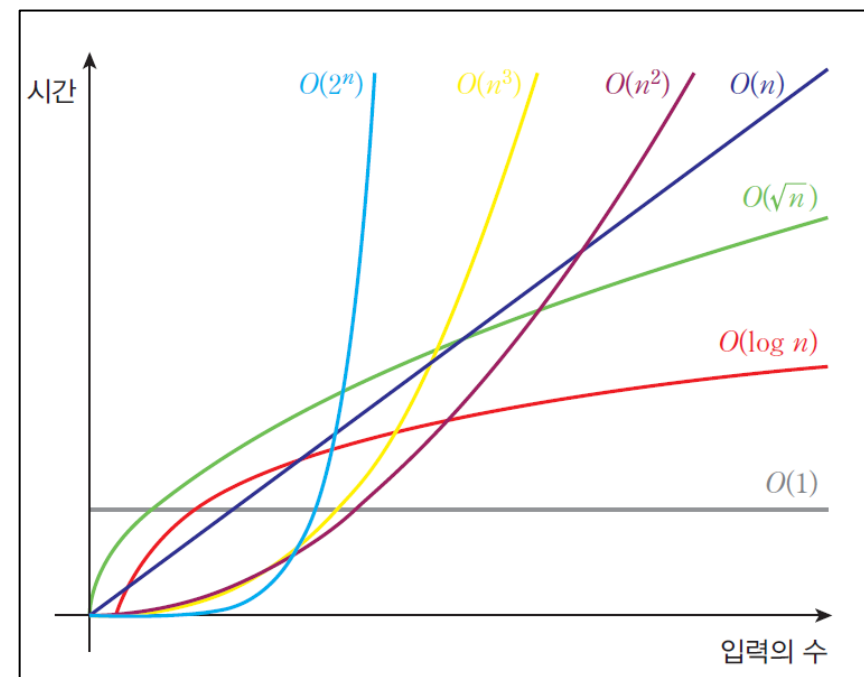
Copy

$\text{gcd}(a, b) \mid \langle \text{numeric} \rangle \mid O(\log n)$

- 두 정수의 최대공약수를 구하는 함수

$\text{lcm}(a, b) \mid \langle \text{numeric} \rangle \mid O(\log n)$

- 두 정수의 최소공배수를 구하는 함수



2. 정수론: 유클리드 호제법

$\gcd(32, 18)$ 은 쉽게 암산으로 풀이가 가능하다.

그렇다면 $\gcd(12345, 123)$ 을 암산으로 구할 수 있을까?
암산이 아니더라도 어디서부터 계산해야 할지 감이 잡히지 않는다.

더 나아가 큰 수들의 $\gcd$ 를 효율적으로 계산하는 방법이 있을까?

→ 유클리드 호제법

2. 정수론: 유클리드 호제법

유클리드 호제법(유클리드 알고리즘)

- 두 정수 A, B 가 있을 때 (단, $A > B$)
- $A = qB + r$ 이라는 식을 세울 수 있다. ($A \bmod B = r$ 과 같음)
- 이때 $\gcd(A, B) = \gcd(B, r)$ 이다.
- $B = 0$ 이 될때까지 반복하면 된다.

예제 1 : $\gcd(24, 10)$

- $\gcd(24, 10) \rightarrow 24 \bmod 10 = 4$
- $\gcd(10, 4) \rightarrow 10 \bmod 4 = 2$
- $\gcd(4, 2) \rightarrow 4 \bmod 2 = 0$
- $\gcd(2, 0) \rightarrow B$ 가 0이 되었으므로 종료, A 에 남아있는 2가 최대공약수

2. 정수론: 유클리드 호제법

예제 2 : $\gcd(12345, 123)$

- $\gcd(12345, 123) \rightarrow 12345 \bmod 123 = 45$
- $\gcd(123, 45) \rightarrow 123 \bmod 45 = 33$
- $\gcd(45, 33) \rightarrow 45 \bmod 33 = 12$
- $\gcd(33, 12) \rightarrow 33 \bmod 12 = 9$
- $\gcd(12, 9) \rightarrow 12 \bmod 9 = 3$
- $\gcd(9, 3) \rightarrow 9 \bmod 3 = 0$
- $\gcd(3, 0) \rightarrow B$ 가 0이 되었으므로 종료, A에 남아있는 3이 최대공약수

예제 3 : $\gcd(20260512, 2026)$

2. 정수론: 유클리드 호제법

유클리드 호제법 | $O(\log n)$

- 재귀 함수를 사용해 두 정수의 최대공약수를 구하기

```
1  #include <iostream>
2
3  using namespace std;
4
5  long long gcd(long long a, long long b) {
6      if(b == 0) return a;
7
8      return gcd(b, a % b);
9  }
10
11 int main() {
12     long long a, b, result;
13
14     cin >> a >> b;
15
16     result = gcd(a, b);
17
18     cout << result;
19
20     return 0;
21 }
```

Input:

20260226 2026

Copy

Expected Output:

2

Copy

2. 정수론: 실습



BOJ 실습

- [5086 배수와 약수](#)
- [13241 최소공배수](#)

2. 정수론: 실습



정올 실습

- [1071 약수와 배수](#)
- [1658 최대공약수와 최소공배수](#)

2. 정수론: 소수 판별

소수(Prime Number)

- 1보다 큰 자연수 중 1과 자기 자신만을 약수로 가지는 수 (ex. 2, 3, 5, 7, 11, 13 ...)
- 즉, 약수의 개수가 2개인 수

소수 판별

- $2 \sim \sqrt{n}$ 까지 나머지 연산을 진행하여 순회가 끝날 때까지 나머지가 0인 케이스가 없다면 소수
- ex) 17의 경우 1~4까지 나머지 연산을 진행함.

2. 정수론: 소수 판별

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7      bool check = true;
8
9      cin >> n;
10
11     if(n <= 1) check = false;
12
13     for(int i = 2; i * i <= n; i++) {
14         if(n % i == 0) {
15             check = false;
16
17             break;
18         }
19     }
20
21     if(check) cout << n << " is Prime Number";
22     else cout << n << " is not Prime Number";
23
24     return 0;
25 }
```

Input: [Copy](#)

Expected Output: [Copy](#)

2 ~ $\sqrt{n}$ 까지 순회를 통해 소수 판별 | $O(\sqrt{n})$

- 약수는 한 쌍으로 존재함($16 = 1 * 16 = 2 * 8 = 4 * 4$)
- 두 인수가 같이 $\sqrt{n}$ 보다 클 수 없음.

2. 정수론: 실습



BOJ 실습

- [1978 소수 찾기](#)
- [4134 다음 소수](#)

2. 정수론: 실습



정올 실습

- [2811 소수와 합성수](#)
- [2813 소수의 개수](#)

2. 정수론: 에라토스테네스의 체

for문을 통한 소수 판별의 한계

- 시간 복잡도가 $O(\sqrt{n})$ 이므로, n 값으로 큰 수(ex. 10^{18})를 계산해야 할 경우 시간 초과 가능성 높음.
- 다수의 입력값을 판별해야 하는 경우 $O(n \sqrt{n})$ 으로 변함

에라토스테네스의 체(Sieve of Eratosthenes)

- 정의 : 자연수 n 이하의 모든 소수를 빠르게 찾아내는 방법
- 1. 초기화 : 2부터 n 까지의 모든 숫자를 나열
- 2. 선택 : 아직 지워지지 않은 숫자 중 가장 작은 수(p)를 찾기(p 는 소수)
- 3. 제거 : p 를 제외한 p 의 모든 배수 지우기
- 4. 반복 : 다음으로 지워지지 않은 숫자를 찾아 2~3번을 반복
- 5. 종료 : $p^2 > n$ 이면 중단하고, 이 때 남은 숫자들은 모두 소수

2. 정수론: 에라토스테네스의 체

2의 배수 제거



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

3의 배수 제거



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

2. 정수론: 에라토스테네스의 체

5의 배수 제거



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

7의 배수 제거



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

2. 정수론: 에라토스테네스의 체

```
1  #include <iostream>
2
3  using namespace std;
4
5  const int MAX = 100000;
6  bool is_prime[MAX + 1];
7
8  int main() {
9      // 전부 true로 초기화
10     for(int i = 2; i <= MAX; i++) {
11         is_prime[i] = true;
12     }
13
14     // 소수가 아닌 수 걸러내기
15     for(int i = 2; i * i <= MAX; i++) {
16         if(is_prime[i]) {
17             for(int j = i * i; j <= MAX; j += i) {
18                 is_prime[j] = false;
19             }
20         }
21     }
```

에라토스테네스의 체 | $O(n \log \log n)$

- if(is_prime[i]) : 아직 지워지지 않은 숫자 중 가장 작은 수(p)를 찾기(p는 소수)
- for(int j = i * i; j <= MAX; j += i)
 - j = i * i부터 시작하는 이유? : i * i 이전의 배수들은 이미 false로 변경 됨.
 - ex) 10, 15, 20은 각각 2, 3에서 처리 됨
 - j += i인 이유? : 배수만 제거하기 위해서

2. 정수론: 에라토스테네스의 체

```
23  int a, b;  
24  
25  cin >> a >> b;  
26  
27  for(int i = a; i <= b; i++) {  
28      if(is_prime[i]) {  
29          cout << i << " ";  
30      }  
31  }  
32  
33  return 0;  
34 }
```

에라토스테네스의 체 | $O(n \log \log n)$

- 약수는 한 쌍으로 존재함($16 = 1 * 16 = 2 * 8 = 4 * 4$)
- 두 인수가 같이 $\sqrt{n}$ 보다 클 수 없음.

Input: Copy
1 100

Expected Output: Copy
**2 3 5 7 11 13 17 19 23 29 31 37 41
43 47 53 59 61 67 71 73 79 83 89
97**

2. 정수론: 실습



BOJ 실습

- [1929 소수 구하기](#)

2. 정수론: 실습



정올 실습

- [1232 에라토스테네스의 체](#)

2. 정수론: To Be Continued...



- 오일러 피 함수
 - 확장 유클리드 알고리즘
 - 중국인의 나머지 정리
 - 밀러-라빈 소수 판별법
-
- 아마도... 여름 방학 중에?